We can simulate the pipeline by passing the Base64 string and checking whether the output is what we expect (it should spit out "Sunday Lunch at Konda's"). The following code tests the pipeline in action. Note that `U3VuZGF5IEx1bmNoIGF0IEtvbmRhJ3M=` is the Base64-encoded PDF file with a secret message ("Sunday Lunch at Konda's").

Listing B.5 Simulating the pipeline

```
POST _ingest/pipeline/confidential_pdf_files_pipeline/_simulate
{
  "docs": [
    {
      "_source": {
        "op_name": "Op Konda",
        "secret_file_data":"U3VuZGF5IEx1bmNoIGF0IEtvbmRhJ3M="
      }
    }
  ]
}
```

The `secret_file_data` field value is manually set with the Base64-encoded string, which is then fed to the pipeline.

Base64 encoding

Java has a `java.util.Base64` encoder class; similarly, Python has a `base64` module. There's most likely Base64 support in your choice of programming/scripting language. Note that we must convert the input into bytes before Base64 encoding can be applied.

We can use any programming language or scripting framework to locate and load files, convert them into bytes, and feed them to the processor. Listing B.5 includes a code snippet for the Java and Python languages. The full code is available with the book's files: Python (http://mng.bz/6D46) and Java (http://mng.bz/o1Zv).

Based on the pipeline definition (listing B.4), the `attachment` processor expects the `secret_file_data` field with the encoded data, and we provided it when simulating the pipeline. Here is the response from the test:

```
...
      "doc": {
        "_index": "_index",
        "_id": "_id",
        "_version": "-3",
        "_source": {
          "op_name": "Op Konda",
          "category": "confidential",
          "attachment": {
            "content_type": "text/plain; charset=ISO-8859-1",
            "language": "et",
```